Claude Overload



or: How I Learned to Stop Worrying and Love the Code Agent

Part II

Colin Swaney, March 20, 2026

Demo: https://github.com/princeton-ddss/data-analyst/tree/dev

Join DDSS Slack to join #claude-workshop channel



Claude Anatomy

Essential Features

1. **Memory** stores essential project context
2. **Skills** supply instructions for repeated tasks
3. **Tools** allow action and communication with the outside world



4. **Permissions** regulate actions and support automation via *rulesets*
5. **Hooks** regulate actions and support automation via *deterministic code*

Claude Anatomy

Context Loading

1. **Startup** memory; system tools; skill, tool, and agent *definitions*

2. **Just-in-time** auto-memory; skill, tool, and subagent *instructions*

3. **Context-free** permissions, hooks

Skills

Don’t Repeat Yourself

- Skills are defined in SKILL.md files injected into context at user request or Claude-initiated
- YAML front-matter control invocation behavior, model use, and tool permissions
- Should instruct Claude how to perform a task or workflow, e.g., summarize-article
- Contextual arguments are determined from message history

`Locations`

`~/.claude/skills` - user-level skills
`.claude/skills` - project-level skills

`Commands`

`/skills` - list available skills
`/<skill_name>` - manually invoke a skill

`Front-matter`

`name` - slash command name
`description` - when to use it (for auto-invocation)
`context` - fork = runs in an isolated subagent
`model` - model to use
`allowed-tools` - comma-separated tools allowed without permissions

Skills

Details & Best Practices

- **Bundled skills:**
  - /batch <prompt> - Orchestrate large scale changes via agents
  - /loop [interval] <prompt> - Run prompt on a loop (e.g., polling)
  - /simplify [focus] - Simplify code with optional emphasis
- Let Claude write functions to minimize randomness
- Focus on **domain-specific knowledge**, team/individual preferences, and manual workflows
- Specify **acceptance criteria** and checkpoints

You can bundle skills functions with skills (and Claude can write them).



Tools

Get Things Done

- Tools define Claude's **core capabilities** and provide access to an external systems
- Tool use requires all function signatures and descriptions to be included in context
- Model returns name of method and arguments—*no guarantee of success*
- Definitions naturally constrain Claude's action space—tools are *safety* features



Built-in Tools

Claude's Core Capabilities

Filesystem

`Read` - read contents of a file
`Write` - create a file
`Edit` - edit a file

Web

`WebSearch` - run a Google search
`WebFetch` - fetch a specific URL

Execute

`Bash` - run commands

Orchestration

`Skill` - run commands
`Task` - run commands

Model Context Protocol

Claude's Extended Capabilities

- MCP servers provide Claude with additional tools, e.g., Slack, Google Drive, Postgres, Context7
- MCP servers tell Claude what's possible. Claude decides what to do. Tool results enter the agentic loop.
- Claude uses **tool search** to reduce the context cost of tool definitions—full JSON schema load is deferred until the tool is needed
  - JSON schema includes parameters, types, required fields, constraints, etc.
- **Example**: https://github.com/slackapi/slack-mcp-plugin

Hooks

Claude's Autonomic System

- Hooks automatically perform an **action** in response to an **event**
- Useful for automation, e.g., always format my `R` code after editing
- But also for guardrails and logging
- Usually run a shell command, which reduces unnecessary context and token usage

Events

`PreToolCall` - Before a tool call executes
`PostToolUse` - After a tool call executes
`PostToolUseFailure` - After a tool call fails
`PreCompact` - Before context compaction
`PostCompact` - After context compaction
`Session Start` - Session beings or resumes
`Notifications` - Claude sends a notification
...

Actions

`Command` - Run a shell command
`HTTP` - Send POST request to URL
`Prompt` - Single-turn LLM evaluation
`Agent` - Spawn a subagent

Hooks
Details

- Hooks live in `settings.json` under the `hook` key
- Scoped to user, project, or local
- Use `matcher` to narrow hooks scope, e.g., only run on `Edit` tool calls
- Hooks can block actions and inject messages into context

Events

`PreToolCall` - Before a tool call executes
`PostToolUse` - After a tool call executes
`PostToolUseFailure` - After a tool call fails
`PreCompact` - Before context compaction
`PostCompact` - After context compaction
`Session Start` - Session beings or resumes
`Notifications` - Claude sends a notification
...

Actions

`Command` - Run a shell command
`HTTP` - Send POST request to URL
`Prompt` - Single-turn LLM evaluation
`Agent` - Spawn a subagent

Claude can perform actions many ways—blocking a single action is rarely enough.



Demo

Closing Thoughts

Learn Git

Just do it

- Claude is powerful, but it needs a *safe* space to work
- GitHub Desktop is an easy way to get started—and GitHub is powerful!
- Claude is a Git wizard—ask Claude to teach you



You with Git



You without Git

Version control project memory, skills, and permissions.



Learn to Delegate

You are the bottleneck

- Code agents work much faster than you
- Agents never get tired—you cannot keep up
- Given a chance—and the right setup—Claude is capable of great things
- You must learn what and how to delegate and review
- **Your new job** is organizing, coordinating, and driving projects



Learn to Flow

1

Pair Programming

- Step-by-step development with tight supervision
- Try planning, but anticipate iterating to get it right

2

Planning Commission

- Construct a detailed plan including checkpoints
- Verify (test) before accepting

3

Autopilot

- Give broad instructions
- Automatically test and auto-accept edits

Summing Up

- Anyone can use Claude Code!
- It’s mostly writing instructions and task management
- Start wherever you feel comfortable
- Bring in tooling as you see fit—you don’t have to optimize
- Ask Claude for help!



End